PHYB57 · Python 速查表（中文人话版）

绿色 = 大白话解释 · 深色 = 代码本身 · 变量名 v / i / x / val / n 都是随便起的名字，可换

1. 基础与运算符 Basics

```
+ 加   - 减   * 乘   / 除(带小数)
// 整除(丢掉小数)   % 取余   ** 次方
7/2 → 3.5     7//2 → 3     7%2 → 1     2**3 → 8
```

运算顺序：先 `( )` → `**` → `* / // %` → `+ -`；再比较，最后逻辑。

```
==  等于    !=  不等于    <  小于    >  大于
<=  ≤    >=  ≥     (= 是赋值，== 才是判断相等)
and 且(都真才真)   or 或(有真就真)   not 非(取反)
```

真假：`True`=1，`False`=0；任何**非 0**的数都算"真"。`(3>2)+(5>4) → 2`

```
int 整数   float 小数   complex 复数
bool 真假   str 文本    type(x) 看类型
2+5j 复数，j 是虚数单位     1e9 =10的9次方
```

```
import math 导入数学库
math.sqrt 开平方   .sin/.cos 三角(用弧度)
.exp e的次方   .log 自然对数   .pi 圆周率
```

2. 变量与赋值 Variables

```
x = 1    把右边的值存进左边(不是"等于")
x += 1   = 先算再加回自己(还有 -= *= /=)
y = x + 1   先算出 y；之后再改 x 不影响 y
```

起名：字母/数字/`_`，开头必须是字母或 `_`，**区分大小写**；别用 `len sum int help` 这些内置名。

```
%whos 看所有变量    del x 删一个    %reset 全清空
```

3. 字符串 Strings

`'…'` / `"…"` 引号包起来的一串字符。索引**从 0 开始**，`-1` 是最后一个。

```
s = "Hello World!"
len(s) 数长度    s[4] → 'o'    s[-1] → '!'
s[0:4] → 'Hell'    取到第4个就停(不含4)
s[::-1] 倒过来    s[0:8:2] 隔一个取
```

```
+ 拼接    str(x) 转文本    x in s 在不在
.upper() 大写   .lower() 小写   .replace(a,b) 替换   .split() 切分
```

f-string：`f"{x}**{y}={x**y}"` **把值/算式直接插进文本**。

4. 列表 Lists [ ] 有序·可改

```
l = [1, 2, 'a', [3,4]]   可混搭/嵌套
l[i] 取第i个   l[-1] 最后一个   len(l) 长度   l1+l2 拼接
.append(x) 加到最后   .insert(i,x) 插到i位
.remove(x) 删第一个   .pop(i) 取出第i个   del l[i] 删
l[i]=x 改   .reverse() 倒序   x in l 在不在
list("abc") → ['a','b','c'] 拆成字符
```

5. 元组 Tuples ( ) 有序·不可改

```
t = (1, 2, 3)    single = (4,)   单个要加逗号
t[i]   t[1:4]   len(t)   .count(x)   t1+t2 同列表
a, b, c = t    解包：一次分别赋给 a,b,c
x, y = (10, 20) 常用来存坐标这种配对
```

改它的元素会报错 `TypeError` **(不可变)**。

6. 集合 Sets { } 无序·去重

```
s = {1,2,3}    set([1,2,2,3]) → {1,2} 自动去重
.add(x) 加    .remove(x) 删
a | b 并集(合起来)    a & b 交集(共有)
a - b 差集(只在a)    a ^ b 对称差(只在一方)
.issubset(b) 子集    .issuperset(b) 父集
```

元素不能是列表 **(要不可变)**，不能靠下标取值。

7. 字典 Dicts {键:值} 无序

```
d = {'a':1, 'b':2}   靠键找值
d[k] 取值   d[k]=v 加/改   del d[k] 删
.keys() 所有键   .values() 所有值   .items() 键值对
len(d) 长度   k in d 键在不在   dict([(k,v)]) 元组转字典
```

键要唯一且不可变(不能是列表)，值随便什么类型都行。

8. NumPy 数值计算

```
import numpy as np   导入并起小名 np
a = np.array([1,4,3])    列表→数组
m = np.array([[1,4],[9,2]])   二维(嵌套列表)
.shape 形状(几行几列)   .size 元素总数
np.arange(起,止,步长)   np.linspace(起,止,个数)
a[1] 第1个   m[0,1] 第0行第1列
a + 2 每个都加2(广播)    a * b 对应位置相乘
```

9. 函数与 Lambda

```
def f(a, b, g=9.81):   定义函数；g 有默认值
    return a + b * g   返回结果(冒号+缩进)
p, q = f(1, 2)   返回多个=一个元组，再解包
```

作用域：函数里的变量只在里面有效；要改外面的全局变量得写 `global n`，否则报 `UnboundLocalError`。

嵌套函数：内层能读到外层变量，但只能被外层调用。

```
square = lambda x: x**2   一行匿名函数，只一个表达式，不用return
sorted(lst, key=lambda t: t[1]) 按第2个元素排序
map(f, seq) 逐个处理   filter(f, seq) 逐个筛选
```

函数是对象，可以当参数传 **(组合 f(g(x)))**。

10. 条件语句 if/elif/else

```
if 条件:        如果(真就执行)
    ...
elif 条件:      否则如果
else:           否则
```

从上往下判断，**第一个真的执行完就退出**。else 只能有一个，elif 可以多个。

三元：`x = a if 条件 else b` **条件真取 a，假取 b**。

11. 循环 Loops

```
for v in seq:        挨个取元素(v是随便起的名字)
for i in range(n):     i 从 0 到 n-1
for i in range(a,b,step): 从a到b-1，步长
for idx, v in enumerate(seq): 同时拿下标+值
while 条件:          条件为真就一直重复
    break     跳出循环
    continue 跳到下一次
```

for = 次数已知；**while** = 次数未知。**while 的条件必须能变假**，否则死循环（`Ctrl/Cmd+C` 中断）。

12. 推导式 Comprehensions

```
[expr for x in seq if cond]   一行生成列表
{k:v for x in seq}              生成字典
{expr for x in seq}             生成集合
[n for n in range(10) if n%2==0] 取偶数
```

= 循环+条件压缩成一行：先遍历，满足条件就放进结果。

13. 复杂度与计时 Big-O

Big-O = 算法随数据量 n 增长的快慢；只留最高次项、忽略常数： `4n²+n+1 → O(n²)`。

```
O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ)
越靠右越慢
嵌套循环 → n²    每次减半 → log n
递归斐波那契 → 2ⁿ    迭代 → n
```

NP-complete：没找到多项式解法的难题(TSP、集合覆盖)。

```
%time 测一行   %%time 测整格   %timeit 多次取平均
cProfile 函数报告   %lprun 逐行计时
```

14. Jupyter 与 pip

```
jupyter notebook 启动    Shift+Enter 运行单元格
Code 代码格 / Markdown 文字格；关标签页≠关服务
```

```
pip install pkg    安装    pip install pkg==1.5 指定版本
pip install -U pkg 升级   pip uninstall pkg 卸载
pip list 列出已装   pip show pkg 看信息
python setup.py install 从源码装
```

`TAB` 自动补全 · `func?` 看说明。